

POLYTECH LYON – MAM4 – ANALYSE NUMÉRIQUE AVANCÉE

Atelier Intensif : Krylov sans matrice pour le Deep Learning

Calcul des valeurs propres dominantes du Hessien (Durée : 6h)

1 Contexte et Rappels Mathématiques

L'objectif est d'analyser la courbure locale de la fonction de perte $\mathcal{L}(\theta)$ d'un réseau de neurones entraîné, via l'étude des valeurs propres de sa matrice Hessienne $H(\theta) = \nabla^2 \mathcal{L}(\theta) \in \mathbb{R}^{n \times n}$. Pour un réseau, n (nombre de paramètres) est trop grand pour stocker H . Nous utiliserons une approche *matrix-free*.

1.1 Le Produit Hessien-Vecteur (HvP)

La différentiation automatique permet de calculer le produit Hv pour tout $v \in \mathbb{R}^n$ sans former H . La technique repose sur l'identité (Pearlmutter [1]) :

$$H(\theta)v = \nabla_{\theta}(\langle \nabla_{\theta} \mathcal{L}(\theta), v \rangle).$$

Interprétation : C'est le gradient de la dérivée directionnelle de la perte dans la direction v . Cela requiert deux passes de rétropropagation (coût $\mathcal{O}(\text{coût gradient})$).

1.2 Algorithme de Lanczos Symétrique (Version de base)

Pour approximer les valeurs propres extrêmes d'une matrice symétrique H , Lanczos construit une base orthonormée d'un espace de Krylov. Soit q_1 un vecteur aléatoire unitaire. Pour $j = 1, \dots, m$:

1. Calculer $w = Hq_j$ (via **HvP**).
2. $\alpha_j = q_j^T w$.
3. $w = w - \alpha_j q_j - \beta_j q_{j-1}$ (avec $\beta_1 q_0 = 0$).
4. $\beta_{j+1} = \|w\|_2$. Si $\beta_{j+1} \approx 0$, stop.
5. $q_{j+1} = w / \beta_{j+1}$.

Les valeurs propres de la matrice tridiagonale T_m (diagonale α_i , sous-diagonale β_i) approximent celles de H .

Objectifs de la séance (6h)

- Implémenter le produit Hessien-vecteur via l'autodiff de PyTorch ou MATLAB.
 - Implémenter l'algorithme de Lanczos unitaire opérant sur cet opérateur HvP.
 - Mettre en évidence le « gap spectral » (quelques valeurs propres très grandes séparées du bulk proche de 0) sur un réseau entraîné sur MNIST, conformément à la littérature [2, 3].
-

2 Travail Demandé (Chronologie indicative)

L'outil recommandé est Python/PyTorch pour la facilité d'accès aux graphes de calcul, ou MATLAB (Deep Learning Toolbox).

2.1 Partie 1 : Implémentation et Validation (2h30)

1. **Modèle** : Charger un Multi-Layer Perceptron (MLP) simple (ex : $784 \rightarrow 100 \rightarrow 10$) pré-entraîné sur MNIST (ou l'entraîner rapidement sur 2-3 époques). Fixer les paramètres θ (mode évaluation).
2. **HvP** : Écrire une fonction `hessian_vector_product(model, loss_fn, data, v)` qui implémente l'identité de Pearlmutter en utilisant `torch.autograd.grad` (ou `dlgradient` sous MATLAB). Veiller à ce que le graphe de calcul du premier gradient soit conservé pour le second.
3. **Validation Cruciale** :
 - Créer un micro-réseau ($n \approx 50$ paramètres).
 - Calculer explicitement H par boucles de gradients (chaque colonne j de H est $\nabla^2 \mathcal{L} \cdot e_j$).
 - Vérifier que `HvP(..., v)` donne le même résultat que $H \times v$ pour un vecteur aléatoire. **Ne pas passer à la suite sans cette validation.**

2.2 Partie 2 : Lanczos et Analyse Spectrale (2h30)

4. **Lanczos** : Implémenter l'algorithme de Lanczos (section 1.2). La fonction prend en argument l'opérateur HvP défini précédemment, le nombre d'itérations m , et un minibatch de données (requis pour calculer $\mathcal{L}$ et ses dérivées).
5. **Calcul** : Lancer Lanczos pour $m = 30$ à 50 itérations sur le Hessien calculé sur un minibatch large (ex : 500 ou 1000 images). Construire T_m , calculer ses valeurs propres (via NumPy/MATLAB).
6. **Analyse** :
 - Tracer les valeurs propres obtenues. Observe-t-on des outliers ?
 - Combien de valeurs propres semblent se détacher du "bulk" ? Faire le lien avec le nombre de classes de MNIST ($C = 10$) [3].

2.3 Partie 3 : Restitution (1h)

Interprétation des résultats et préparation des livrables.

Livrables fin de séance :

- Code source fonctionnel et validé.
- Une figure montrant le spectre obtenu (Ritz values) et la courbe de convergence de $\lambda_{\max}$ en fonction de m .
- Discussion orale (5-10 min) sur les difficultés rencontrées (notamment l'orthogonalité dans Lanczos) et l'interprétation du gap spectral observé.

Références

- [1] B. A. Pearlmutter. *Fast exact multiplication by the Hessian*. Neural Computation, 1994.
- [2] L. Sagun et al. *Empirical analysis of the Hessian of over-parametrized neural networks*. arXiv :1706.04454, 2017.
- [3] V. Pappayan. *The full spectrum of deepnet Hessians at scale*. arXiv :1811.07062, 2018.